

PART 1: LITERATURE SURVEY TABLE

11 of 16 papers made the cut for two slides, ranked by relevance to TRO. Exclusion rationale is below the table — nothing was silently dropped.

Slide 1 of 2

Paper	Objective	Methodology	Pros/Cons	Findings
TRIANGLE (Multi-Agent Incident Triage, ASE 2025)	Automate end-to-end incident triage — assigning cloud incidents to the correct team — via specialized, negotiating LLM agents, tested in real production and public benchmarks.	Multi-agent LLM framework: incident semantic distillation, multi-role agents with a negotiation protocol, automated Team Information Enrichment; evaluated on real production incidents (tens of millions of users, 20TB/day, p.6) plus the MSR Bug Dataset.	Pro: up to 97% triage accuracy, 91% engagement-time reduction at real production scale. Con: "A common failure mode we observed stemmed from missing or insufficient historical signals... essential for associating incidents with the correct team" (p.9).	Negotiating multi-agent LLMs substantially beat single-model/rule-based triage at real scale — but the entire system operates only after an incident record already exists.

TickIt (ByteDance, FSE 2025)	Automate online ticket-escalation decisions — when to escalate, to whom, whether it duplicates an existing ticket — using LLM reasoning at production scale.	LLM chain-of-thought classification with category-guided supervised fine-tuning; embedding dedup ($\theta=0.88$ cosine); deployed on Volcano Engine; evaluated on 20,066 tickets / 654,901 messages.	Pro: 39% real-world MTTR reduction; a live feedback→fine-tuning loop. Con: "personalized expressions from different customers can influence the performance of the LLM... a bland description of serious problems could result in TickIt overlooking necessary escalations" (p.10).	LLM escalation + embedding dedup measurably cuts MTTR in live production, but accuracy still depends on how the customer phrased the ticket — after it was already created.
ClarifyMT-Bench (arXiv:2512.21120)	Benchmark whether/when LLMs should ask clarifying questions in multi-turn, adversarial dialogue; propose ClarifyAgent to fix systematic under-clarification.	5-dimension ambiguity taxonomy; 6 personas (incl. Refusal); 6,120 LLM-human dialogues; 10 LLMs evaluated; modular ClarifyAgent (Perceiver, Forecaster, Tracker, Planner, Output).	Pro: ClarifyAgent lifts weaker models sharply (Qwen-2.5-7B 66.4%→88.0%). Con: "ClarifyAgent requires five forward passes of the base LLM (once per module)" (p.8) — real per-turn latency/cost.	Ten off-the-shelf LLMs show a consistent under-clarification bias — answering prematurely, degrading on noisy/contradictory/refusal input — directly motivating a hard turn-cap in any deployed clarifier.

Zangari et al. (Ticket Automation survey, ESWA 2023)	Establish a task taxonomy for Ticket Automation research; benchmark transformer (BERT/XLNet) hierarchical classifiers on real multi-level ticket/bug data.	Literature-derived 4-task taxonomy — Ticket Classification, Expert Finding, Ticket Routing, Ticket Resolution (Table 1, p.5); 3 hierarchical BERT architectures; 2 real datasets; stratified cross-validation.	Pro: rigorous, transparent real-data benchmark; embedding-strategy choice alone shifts F1 by >28%. Con: "the supervised approach may not be the most suitable for real-world applications, since companies may not know the set of target labels" (p.17).	Even strong transformer classifiers plateau at modest macro-F1 (≈ 0.18 – 0.60 , pp.13–14) on real noisy multi-level ticket text — the realistic ceiling TRO's Metric 1 target references.
Towards Asking Clarification Questions (MAS2S)	Address users' inability to fully articulate information needs in task-oriented dialogue by modeling when/what to ask.	MAS2S (Multi-Attention Seq2Seq); new $\sim 100k$ -dialogue ClariT dataset; benchmarked against UBAR using success rate, question count, oracle-distance.	Pro: MAS2S's average clarifying-question count (2.97) tracks much closer to oracle-optimal (2.36) than UBAR (p.8). Con: "this work is just one of the first steps" (p.9) — field-wide immaturity, not just this paper.	Multi-attention modeling narrows the gap to the oracle-optimal number of clarifying questions, but the field is explicitly described by its own authors as still in its "initial stage."

Slide 2 of 2

Paper	Objective	Methodology	Pros/Cons	Findings
-------	-----------	-------------	-----------	----------

AIM Survey (Yu et al., J. Network & Computer Applications 2024)	Comprehensive ly survey 89 papers (2008–2022) on AI-driven alert/incident management; propose a unified pipeline taxonomy and open challenges.	Systematic survey (snowball + keyword search); unifies architectures into one monitoring→alert→incident pipeline; no original dataset/model — a taxonomy-and-synthesis contribution.	Pro: most comprehensive AIM survey to date; confirms tickets originate from severe alerts or user complaints alike. Con: "publicly available benchmark datasets and uniform evaluation approaches are needed to achieve quantitative comparison, which is missing in the literature" (p.16).	"In the ICM system, a reported incident is recorded as a ticket" (p.3) — across all 89 surveyed papers, every AIM technique reviewed operates only after that recording happens.
---	--	--	--	--

Human vs. Automated Data Annotation (Fuchs et al., TUM/SAP, 2026)	Compare human vs. automated (Active Learning + Weak Supervision) annotation for an ML ticket classifier; examine ticket ambiguity's impact on distribution quality.	Case study on real SAP ticket data; two parallel annotation pipelines — crowdsourced human labels vs. algorithmic Learning Functions needing only 10% human-labeled seed data.	Pro: automated pipeline reached 92% agreement with human labels at a fraction of the cost. Con: proposes, then declines, a pre-distribution check: "designing such an ambiguity check is a whole project in itself, we think our solution is more straightforward" (p.9).	The authors explicitly describe a pre-distribution "ambiguity check" NLP model that flags vague tickets and requests more information — then decline to build it, calling it "a whole project in itself" (p.9).
---	---	--	---	---

Fusing Tickets and Alerts (Salah et al., Information Fusion 2019)	Improve network-incident resolution efficiency by fusing human ticket text with machine alert data for correlation, on a real telecom network.	Rule-based time-alignment correlation (time thresholds + node/keyword similarity); validated on real alert/ticket data from a large production telecom network.	Pro: validated on real production data (not synthetic); measurably raises alert-reduction rate. Con: "the similarity and filtering criteria used for alerts and tickets depend on various rules and a list of human-provided keywords that may depend on the considered network" (p.14).	Fusing tickets with alerts measurably improves correlation on real data, but is rule/keyword-dependent and, like all reviewed correlation work, runs strictly on tickets already created.
---	--	---	--	---

Combining Deep Ensemble Learning and Explanation (Zicari et al., ESWA 2022)	Build an accurate <i>and</i> explainable ticket classifier, combining deep ensembles with LIME-style local explanation for human oversight.	Deep ensemble of CNN/FFNN base classifiers; LIME-based local explanation generation; evaluated on real (non-synthetic) ticket data.	Pro: validated on real data; generates genuinely human-facing rationale. Con: "we plan to investigate extending the framework with... Deep Active Learning mechanisms... to allow human experts to provide new informative example tickets" (p.15) — no feedback-to-re training loop exists yet.	Deep ensembles + local explanation measurably improve robustness and trust in ticket routing — showing explainable-rationale generation is already mature outside ITSM-specific gating.
Resolving Intent Ambiguities (Dhole)	Generate discriminative clarifying questions between two competing intents in task-oriented dialogue, without annotated clarification-question data.	Rule-based hybrid of a question-generation model + sentence-similarity model (template + generative); evaluated via human Likert ratings and coverage metrics.	Pro: needs no clarification-labeled training data; decouples cleanly from the intent classifier. Con: "the coverage of the discriminative similarity approach is still low and holds tremendous scope for improvement" (p.6).	Rule-based question generation can meaningfully disambiguate two intents without labeled data, but coverage stays low — direct evidence for why TRO rejects a rule-based question bank.

Scalable Customer Support Ticket Analysis (XLNet/HGB)	Predict ticket priority, resolution status, and customer satisfaction using XLNet embeddings feeding a Histogram-Based Gradient Boosting classifier.	XLNet feature extraction → HGB multi-target classifier; reports 96.26% (satisfaction) / 95.41% (resolution) accuracy; no stated dataset source or leakage discussion.	Pro: direct precedent for TRO's own planned Agent-5 phased upgrade. Con: <i>no self-reported limitation exists anywhere in the text</i> — paired with unusually high accuracy on subjective targets, this is a red flag (my inference, not the paper's claim).	Reports very high accuracy on leakage-prone targets with no discussion of provenance or train/test methodology — treat these specific figures with real caution.
---	--	---	--	--

Excluded from the slide table (fully read, available if you want them added back):

- *A machine learning-based help desk system* (Al-Hawari & Barham) — real SVM deployment, genuinely solid, but substantively overlaps with Zangari's ceiling point.
- *Predicting help-desk reassignments with GCNs* (Schad et al.) — has zero NLP component: "Textual descriptions of ticket details were not available in the dataset used in this work for privacy reasons" (p.8) — most tangential to TRO's language-first design.
- *AI-Powered Complaint Management* (IRJMETS) — conceptual only; every number is self-described as "assumed" (p.5), not measured. Kept for Part 2D below.
- *Online Chatbot's Ticketing System* (IJIRSET) — narrative synthesis of three external case studies, no original methodology, lower-tier venue.
- *Smart Serve* (ASTJ, 2026) — very new/low-citation venue; reports precise metrics for its *pre-pivot* baseline models but zero quantitative metrics anywhere for its own headline Gemini-based system.

PART 2: RESEARCH SYNTHESIS

A. Research Gap

Gap 1 — The entire reviewed literature intervenes only *after* a ticket/incident record already exists.

- Evidence: AIM Survey — *"In the IcM system, a reported incident is recorded as a ticket"* (p.3); TRIANGLE — *"TRIANGLE relies heavily on historical incident triage records... to perform automated triage"* (p.9); Fuchs et al. — describe exactly this pre-creation check, then say *"designing such an ambiguity check is a whole project in itself"* (p.9).
- Why unresolved: three independent groups, three different sub-fields (large-scale AIOps survey, production multi-agent triage, ML-annotation methodology), converge on the same boundary without crossing it.
- **TRO's status: directly targeted, not fully "solved."** This is architectural *repositioning* — TRO doesn't invent a new way to resolve ambiguity/duplication, it relocates *where* existing techniques (LLM clarification, embedding similarity) are applied. That distinction matters for how the paper should frame its contribution — see Part C.

Gap 2 — LLMs systematically under-ask clarifying questions, and this remains unresolved even in specialized clarification research.

- Evidence: ClarifyMT-Bench — *"a consistent under-clarification bias: models tend to answer prematurely, struggle to remain robust under noisy or contradictory user feedback, and degrade substantially as dialogue depth increases"* (p.9); MAS2S paper — *"this work is just one of the first steps"* (p.9), and its own best model still needs 2.97 turns vs. a 2.36 oracle (p.8).
- Why unresolved: even the paper *purpose-built* to fix this (ClarifyAgent) doesn't reach the oracle-optimal turn count and adds real latency (5 forward passes/turn, p.8).
- **TRO's status: partially mitigated, not solved at the root.** Agent 2's confidence-gating + 2–3 turn cap is a pragmatic engineering fix for the *Refusal-persona* failure mode ClarifyMT-Bench documents (p.4-5) — it doesn't claim to fix the underlying model bias.

Gap 3 — Correlation/deduplication techniques rely on hand-tuned, domain-specific rules that don't generalize.

- Evidence: Salah et al. — *"the similarity and filtering criteria used for alerts and tickets depend on various rules and a list of human-provided keywords that may depend on the considered network"* (p.14); TickIt — flags that its own $\theta=0.88$ threshold-based dedup can misfire when tickets *"exhibit the same symptoms but have different root causes"* (p.9).

- Why unresolved: TickIt's own authors note their threshold was empirically swept for their own domain, not assumed transferable — exactly the caveat TRO's own Metric 9 already anticipates.
- **TRO's status: partially mitigated, unvalidated.** Agent 4 swaps rules for embeddings — a real improvement path — but TRO's own proposal document concedes its threshold "is not guaranteed to transfer to TRO's own data distribution," so this remains a design intention, not yet a demonstrated result.

Gap 4 — Reported business-impact figures (cost savings, MTTR) are frequently unvalidated projections, or, when real, require production-scale access a capstone project cannot get.

- Evidence: IRJMETTS paper — explicitly, *"It is assumed that the AI-driven complaint management system will record notable gains in various areas of performance relative to the manual ones"* (p.5) — every headline number (25–40% cost reduction) follows directly from that sentence. Second, indirect confirmation: Smart Serve reports precise precision/recall tables for its discarded baseline models but supplies **no quantitative metrics anywhere** for its own headline Gemini-based system — the same "claims exceed evidence" pattern revealed through absence rather than admission (**this is my inference from what's missing, not a quote**).
- Why unresolved: TickIt shows it *can* be done honestly (39% MTTR reduction on 654,901 real messages) — but that required exactly the kind of production access neither IRJMETTS, Smart Serve, nor TRO's capstone timeline has.
- **TRO's status: already handled correctly.** Your own Metric 10 explicitly refuses to project this number from synthetic data — genuinely better practice than half of this literature set. Worth stating this contrast plainly in the paper; it's a real point in TRO's favor.

B. Research Objective

Across the ticket-automation literature — from rule-based routing through RAG-based deflection to state-of-the-art multi-agent triage (TRIANGLE) — every reviewed system begins its work only after a ticket or incident record has already been instantiated, treating ticket creation as a fixed boundary rather than a decision. This project's research objective is to test whether a confidence-gated, multi-agent architecture applied *upstream* of that boundary — at the point of the raw, unstructured user message — can measurably reduce three costs the reviewed literature either doesn't measure or explicitly declines to address: tickets created from incomplete descriptions requiring manual back-and-forth, tickets created for issues already answered in existing documentation, and near-duplicate tickets from one incident triaged independently. Concretely, the problem solved is deciding, from a single natural-language message and before any ticket record exists, whether a ticket needs to be created at all — and if so, ensuring it is created complete, non-duplicate, and accompanied by a machine-generated rationale a human reviewer can audit.

C. Novel Contribution

Be precise about what's actually new here, because a reviewer will be: **none of TRO's six agents introduce a new machine-learning technique.** LLM-based extraction,

confidence-gated clarification, embedding similarity, LLM classification-with-rationale — all of these exist, individually, in the papers above (ClarifyMT-Bench/MAS2S for gated clarification; TickIt/Salah et al. for embedding correlation; Zicari et al./TickIt's Thinking field for explanation generation). What none of the 16 papers do — confirmed independently across a comprehensive 89-paper survey (AIM Survey, p.3), a production multi-agent system (TRIANGLE, p.9), and a paper that explicitly proposed and declined to build the exact mechanism (Fuchs et al., p.9) — is **apply and sequence these three known mechanisms (completeness gating, KB deflection, temporal correlation) together, upstream of ticket creation, as one evaluated pipeline**. The genuine contribution is architectural positioning and integration, not algorithmic invention. Framing the paper this way — which your own proposal document already does in Section 4 — is the right call and should be foregrounded, not softened.

D. Feasibility Analysis — harsh reviewer pass

1. Is agentic AI actually necessary here, or would a simpler pipeline do?

Your own architecture document already concedes this, refreshingly: Agents 1, 4, and 6 are "closer to classical pipeline stages than autonomous reasoning agents." Only Agents 2, 3, and 5 involve real LLM judgment, and even those are single-shot calls (one question, one retrieval+compose, one classify+rationale) rather than iterative planning/replanning loops. Compare this to what TRIANGLE calls "multi-agent" — specialized roles *negotiating* over multiple rounds. A reviewer versed in that literature will ask where TRO's inter-agent negotiation or tool-selection reasoning is. The honest answer, per your own Section 9, is: deferred to future work. This is fine *if stated up front*; if a reviewer discovers it unprompted, it reads as "agent-washing" a workflow. **Recommendation: lead with the "3 of 6 are agentic, 3 are workflow" framing in the paper's own architecture section, don't let a reviewer find it.**

2. Why might this fail in practice?

- The 2–3 turn cap's fallback ("proceed with partial info, flag lower confidence") is a good mitigation for the Refusal-persona risk ClarifyMT-Bench documents — but nothing in the current 10-metric framework measures what happens to downstream classification/routing accuracy for tickets that take this fallback path specifically.
- Metric 6 (confidence calibration) tests an assumption that **none of the 16 papers reviewed validate for their own systems** — this is the single largest untested premise underlying Agents 2, 3, and 5 all at once. If calibration turns out poor, the entire gating architecture's rationale weakens, and there's no described contingency in the current design.
- Claim #2 (temporal correlation) is ranked your *strongest* novelty claim, yet by your own Metric 5's admission, "a genuinely convincing test set is more work than it looks" — meaning the strongest claim is also the hardest to evaluate credibly on synthetic data alone. That tension deserves explicit acknowledgment in the paper, not just the metrics table.

3. Currently unsupported claims:

The claim that a *unified* upstream pipeline (all three gates together) outperforms doing these things separately or post-creation is not demonstrated by any combination of these 16

papers — none studies more than one or two of these mechanisms together, let alone pre-creation. That's the crux of TRO's novelty, and right now it's an assumption to test, not a supported finding.

4. Experiment most obviously missing: an ablation against a "no-gate" baseline — extract → classify → route directly, no clarification/deflection/correlation — on the *same* synthetic dataset. Nothing in the current plan directly measures whether upstream gating improves final ticket quality versus just skipping to classification. This is the one experiment a reviewer will ask for first, and it's currently absent. **(This recommendation is mine, not drawn from any single paper — it follows from the pattern across all of them.)**

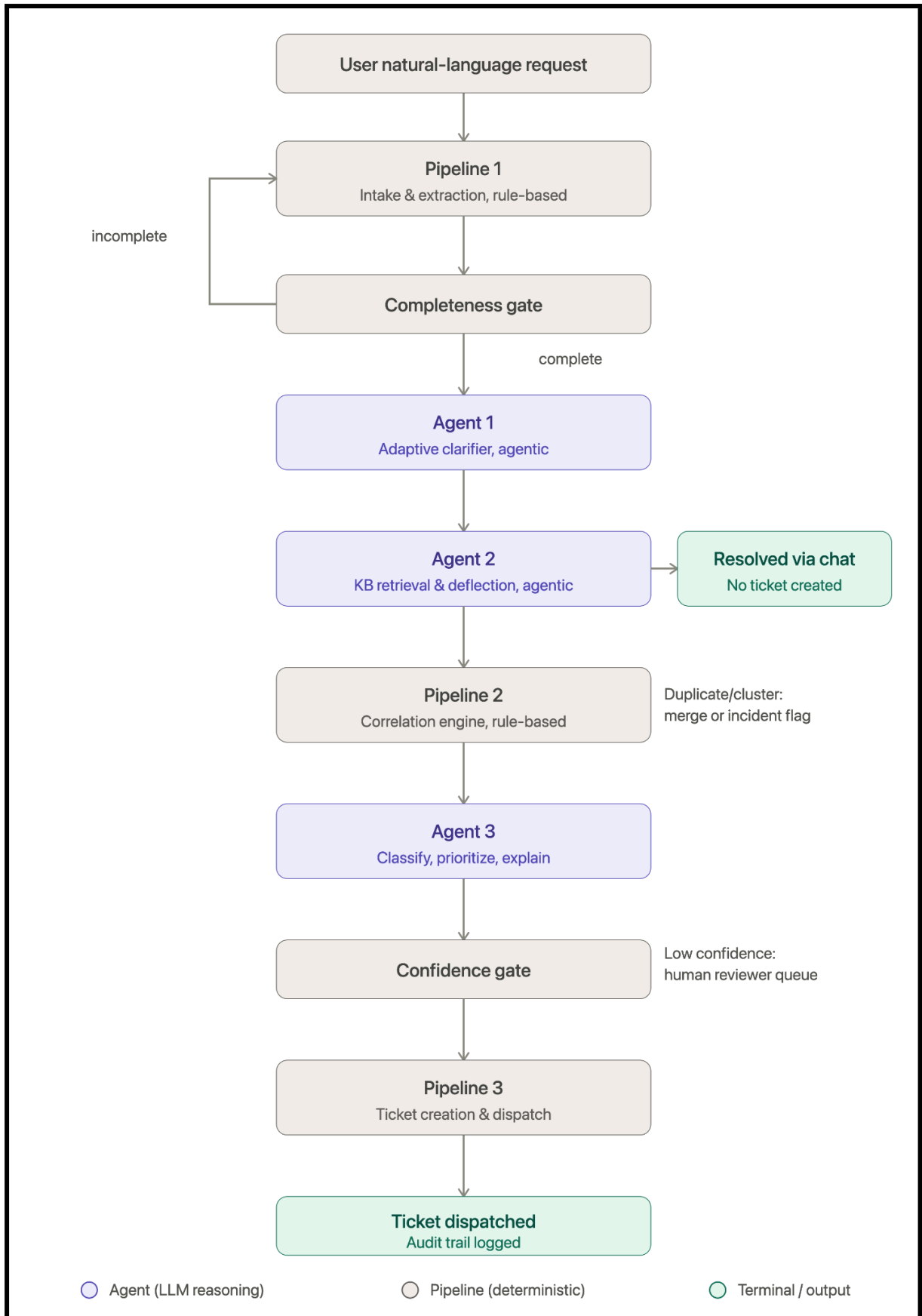
5. Incremental vs. genuinely novel, by agent: Agent 1 (extraction) — incremental, standard NLP. Agent 5's core classifier — incremental (same family as Zangari/XLNet-HGB); the routing_rationale layer is a real but narrow differentiator, matching your own "weakest claim" ranking. Agent 6 — pure integration, not a research contribution, by your own document's admission. Agent 4's mechanism (embeddings + threshold) is technically standard (TickIt already does embedding-based dedup); what's novel is *when* it runs, not *how*.

E. Validation Metrics

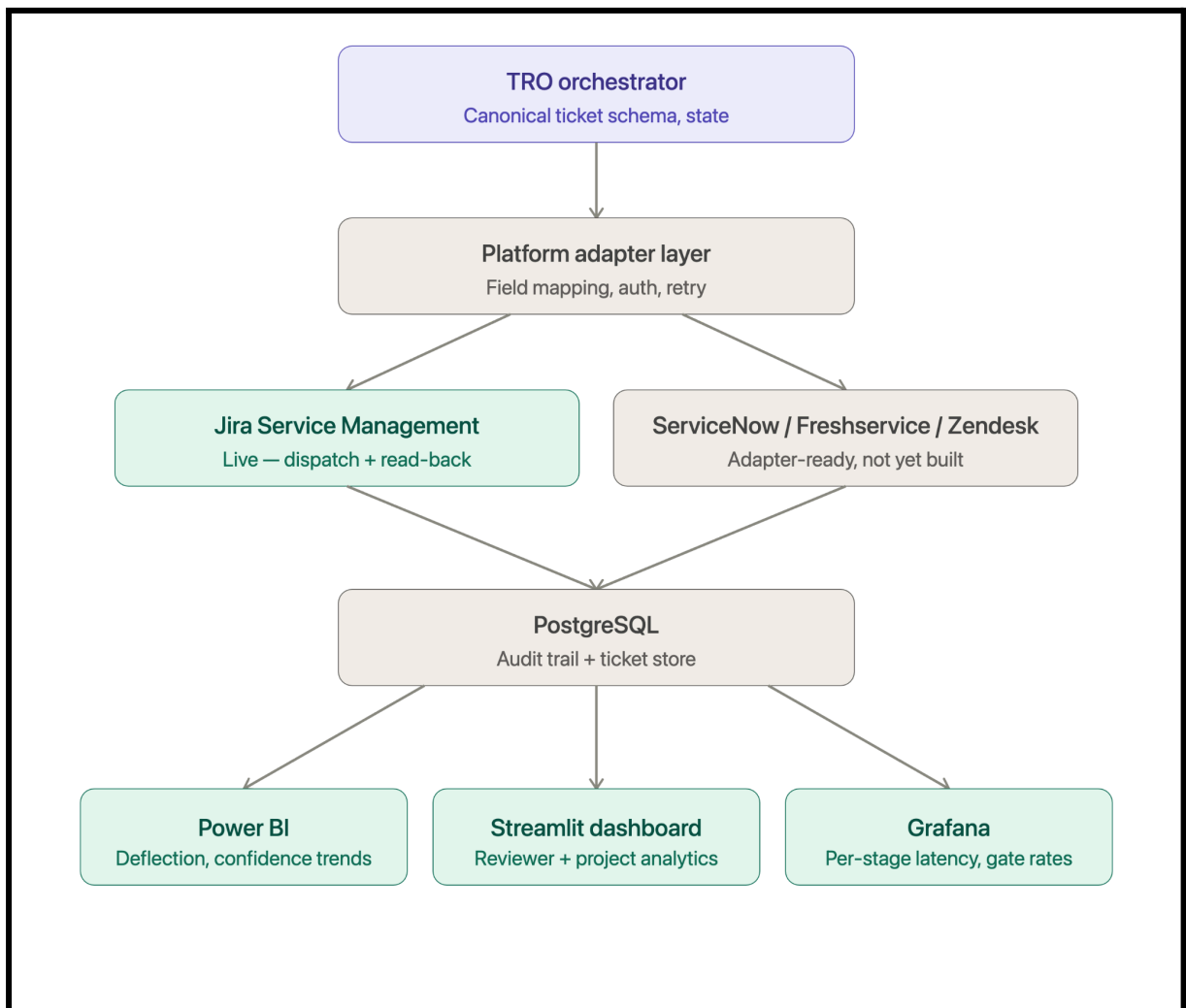
Your existing 10-metric framework is well-designed; here's each tied to a verified literature comparator, plus two additions the feasibility pass above surfaces as missing:

1. **Classification/routing F1** — benchmark against Zangari's own reported ≈ 0.18 – 0.60 macro-F1 range (pp.13-14), not a single number.
2. **Deflection rate + deflection accuracy** — correctly paired; no comparator exists in any of the 16 papers, so this pairing is itself a rigor addition.
3. **Clarification-turn efficiency** — benchmark against MAS2S's 2.97-vs-2.36 oracle gap (p.8); also worth referencing Dhole's coverage framing (p.6) as a secondary comparison point.
4. **Correlation precision/recall** — mirror TickIt's θ -sweep methodology (p.9) rather than assuming a fixed threshold, since no directly transferable baseline exists.
5. **LLM confidence calibration** — genuinely novel; no paper in this set tests it. Given how much of the architecture depends on this assumption (see D.2), **run this first, not sixth.**
6–8. Escalation rate, latency/cost — as planned; TRIANGLE (pp.9-10) independently confirms multi-step LLM latency/cost is a real, cited risk for voting-style architectures.
6. MTTR/FRT/cost savings — correctly disclaimed per Metric 10; contrast IRJMETs's "assumed" figures (p.5) against TickIt's real 654,901-message validation (p.4) in the paper's limitations section, for effect.
7. **[Recommended addition]** Ablation vs. a no-gate baseline on identical synthetic data — tests the core novelty claim directly.
8. **[Recommended addition]** Downstream routing accuracy specifically for tickets that hit the turn-cap fallback path, vs. tickets that reached full completeness normally.

PART 3: ARCHITECTURE



With Jira integration -



PART 4: SPECIFIC TICKET FIELDS

Looking at the three project docs and the reference paper list, a clear pattern emerges: almost every existing dataset and paper (Ticket-BERT, Zangari survey, TickIt, iPACK, the Kaggle ITSM sets) is trained on the same handful of generic buckets — Hardware, Network, general "Software," and Access/password-reset. Those are safe but oversaturated, which actually hurts your novelty claim. The domains below are chosen because they (a) carry real operational weight in MSPs like Kyndryl/TCS/Wipro, (b) aren't deeply modeled in the papers you've listed, and (c) map cleanly onto TRO's 6-agent pipeline (completeness scoring, KB deflection, temporal correlation, explainable routing) without needing exotic data you can't get in 3–4 months.

1. Identity & Access Governance (beyond simple password reset)

Why it's underexplored:

Most "Access" categories in existing datasets/papers collapse everything into one bucket ("Access" or "Account"). None model the actual workflow: role-based access requests, approval chains, entitlement mismatches, segregation-of-duties conflicts. This is a real compliance pain point for MSPs and banks/telecom clients specifically named in your doc.

Why it's achievable:

The schema is well-defined (requester, system, role/entitlement, approver, business justification) — perfect for your completeness-gate and policy-aware validation novelty claims. Jira and ServiceNow both have native approval/workflow fields you can map to, so the adapter layer is straightforward. Synthetic data generation is easy (finite set of apps × roles × approval rules).

TRO fit:

Showcases Agent 1/2 (structured slot-filling), Agent 5 (explainable routing — "routed to IAM team because role X requires manager approval"), and ties naturally to audit trail requirements for PowerBI compliance dashboards.

2. Collaboration & Productivity Suite Issues (Email/O365/Teams/SSO outages)

Why it's underexplored:

Almost no public ticket dataset separates "collaboration tools" from generic "software." Yet in real MSP environments this is one of the highest-volume categories, and it's the best possible showcase for your ****temporal correlation**** novelty claim (Differentiator #2) — when Outlook or Teams SSO breaks, dozens of near-identical tickets land within minutes, which is exactly the incident-clustering behavior TickIt and iPACK don't handle at intake.

Why it's achievable:

You can synthesize realistic bursts of similar tickets easily (same root cause, slightly different phrasing per user), which is exactly what you need to demo Agent 4's Sentence-BERT duplicate detection and incident-cluster flagging convincingly in a viva/demo.

TRO fit:

Best domain to **demo live** — you can literally script 5 tickets in 2 minutes and show the system auto-merge them into one incident instead of creating 5 separate Jira issues.

3. Software/SaaS License & Provisioning Requests

Why it's underexplored:

Papers treat "software issues" as bugs/errors, not the very common (and costly) "I need access to X software" / "renew my license" request type. This has direct, easy-to-articulate business value (cost control, license sprawl) that's rarely framed as a ticket-automation research problem — good for your real-world-application section.

Why it's achievable:

Bounded vocabulary (finite list of tools), clean structured fields (tool name, license tier, cost center, duration), and it's the strongest showcase for ****KB-grounded deflection****

(Differentiator #3) — many of these requests already have a documented self-service path ("use the internal license portal") that the system can surface before ever creating a ticket.

****If you want my honest ranking of the three for a 3–4 month build:**** start with ****Collaboration/Productivity outages**** as your flagship demo domain (best visual "wow" factor for temporal clustering), use ****Identity & Access Governance**** as your secondary domain to show off completeness scoring and explainable routing, and treat ****Software/SaaS Licensing**** as a lighter third category mainly to prove the KB-deflection pipeline works before a ticket is even created. Together the three domains exercise all four of TRO's novelty claims without needing you to build a network-engineering or OS-internals dataset from scratch, which would eat your timeline for little added novelty.

PART 5: OUTPUT

Here are the terminal (final) outputs the pipeline can reach — same four decision paths in both versions, but what actually happens at the last step changes once Jira is in the loop.

Without Jira (current MVP — Pipeline 3 only writes to Postgres)

1. ****Resolved via chat**** — Agent 2's KB match is confirmed by the user. No ticket record of any kind is created; only the conversation and the deflection outcome are logged.

Example message: "Glad that fixed it — no ticket needed. I've logged this as resolved."

2. ****Ticket created internally**** — Pipeline 3 writes a structured ticket into Postgres with a TRO-generated ID, category, priority, and routing rationale. It exists only inside TRO; nothing external is notified.

Example message: "Ticket TRO-2024-00892 created — routed to Network Operations, priority P2."

3. ****Merged / duplicate**** — Agent... no, Pipeline 2 (correlation engine) matches this to an already-open internal ticket. No new record; the existing one gets a linked reference.

Example message: "This looks like the same issue as TRO-2024-00887, filed 40 minutes ago — I've linked yours instead of creating a new one."

4. ****Escalated to human reviewer**** — Confidence gate sends it to the review queue with the full reasoning trace attached. Nothing is dispatched until a reviewer acts.

Example message: "This one needs a quick human check before it's finalized — it's in the review queue now."

With Jira (future enhancement — Pipeline 3 dispatches through the adapter layer)

1. ****Resolved via chat**** — identical to the no-Jira case; Jira is never touched since no ticket is created.

Example message: "Glad that fixed it — no ticket needed."

2. **Ticket submitted to Jira** — the canonical schema is translated by the Platform Adapter Layer and a real Jira issue is created via `POST /rest/api/3/issue`. TRO gets back a Jira key and surfaces it to the user.

Example message: "Ticket ITSD-4521 created in Jira, assigned to Network Operations, priority P2. [link]"

3. **Merged into existing Jira ticket** — correlation engine matches against Jira's live open-ticket pool (via the JQL read-back), not just TRO's internal store, so it can catch duplicates other channels filed directly in Jira too.

Example message: "Three people reported this in the last 90 minutes — I've linked yours to ITSD-4518, which is already being worked."

4. **Escalated to human reviewer** — same as before, but on approval the reviewer's decision is what actually triggers the Jira `POST` call; on rejection, nothing is dispatched at all.

Example message: "Flagged for reviewer sign-off before it goes into Jira."

5. **Dispatch failed / queued for retry** — a Jira-specific failure mode that doesn't exist without Jira: the API call errors (auth, rate limit, network) and the adapter layer's retry logic kicks in instead of silently dropping the ticket.

Example message: "Your ticket is ready but Jira didn't respond — it's queued and will submit automatically once the connection is back."

The practical difference: **the no-Jira version's "ticket created" output is a closed loop entirely inside TRO** (useful for the capstone demo and metrics, but not visible to a real service desk); **the Jira version turns that same output into something an actual support team sees and works**, at the cost of adding a fifth outcome (dispatch failure) that only exists once you're calling a real external API.